



HealthFlow

Manual de Instalación y Configuración

Sistema de Gestión Hospitalaria · v1.0 · 2026

Motor: PostgreSQL 12+ (Supabase) · Backend: FastAPI · CLI: Python 3.12+

v1.0

2026

Supabase

FastAPI



Tabla de Contenido

1

Requisitos Previos

Herramientas necesarias antes de instalar

2

Clonar el Repositorio

Descargar el código fuente

3

Entorno Virtual

Aislar las dependencias del proyecto

4

Variables de Entorno (.env)

Configurar credenciales y parámetros

5

Base de Datos en Supabase

Crear tablas y datos de prueba

6

Ejecutar el Backend FastAPI

Levantar la API REST

7

Ejecutar el CLI de Usuarios

Interfaz de línea de comandos

8

Verificación del Sistema

Comprobar que todo funciona

9

Solución de Problemas

Errores comunes y sus soluciones

1 Requisitos Previos

Antes de instalar HealthFlow, verifica que tienes instaladas las siguientes herramientas:

Herramienta	Versión mínima	Comando de verificación
Python	3.12+	python --version
pip	23+	pip --version
Git	2.40+	git --version
psycopg2-binary	2.9+	pip show psycopg2-binary
bcrypt	4.0+	pip show bcrypt

■ **Tip:** Si usas Windows, descarga Python desde python.org y marca 'Add to PATH' durante la instalación.

2 Clonar el Repositorio

Abre una terminal y ejecuta:

```
$ git clone https://github.com/tu-usuario/health-flow-proyecto.git

$ cd health-flow-proyecto

$ git checkout develop
```

Estructura del repositorio:

```
health-flow-proyecto/

■■■ backend/ # API REST con FastAPI

■ ■■■ app/

■ ■ ■■■ auth/ # Módulo de autenticación (login, register, users)

■ ■ ■■■ main.py # Punto de entrada de la aplicación

■ ■ ■■■ database.py # Configuración de SQLAlchemy
```

```
■ ■ ■■ config.py # Variables de entorno con Pydantic

■ ■■ .env # Credenciales (NO subir a Git)

■ ■■ requirements.txt

■■■ scripts/

■ ■■ crud.py # CLI de gestión de usuarios

■■■ sql/

■ ■■ ddl/ # Scripts DDL (creación de tablas)

■ ■■ dml/ # Scripts DML (datos de prueba)

■ ■■ queries/ # 10 consultas avanzadas de demostración

■■■ docs/ # Documentación del proyecto
```

3 Entorno Virtual

El entorno virtual aísla las dependencias del proyecto del sistema operativo. Sigue los pasos según tu sistema operativo:

Windows

```
# Desde la carpeta raíz del proyecto

$ cd backend

$ python -m venv venv

$ venv\Scripts\activate

# Instalar dependencias

$ pip install -r requirements.txt
```

macOS / Linux

```
$ cd backend

$ python3 -m venv venv

$ source venv/bin/activate

# Instalar dependencias

$ pip install -r requirements.txt
```

■ **Tip:** Si no tienes requirements.txt, instala manualmente:

```
pip install fastapi uvicorn sqlalchemy psycopg2-binary python-jose passlib bcrypt pydantic[email] pydantic-settings
```

4 Variables de Entorno (.env)

Crea el archivo **backend/.env** con el siguiente contenido. Este archivo contiene credenciales sensibles y **nunca debe subirse a GitHub** (está incluido en .gitignore).

[illegible]

■ **Aviso:** Reemplaza TU_PASSWORD y TU_REF con los valores reales de tu proyecto Supabase.
Los encuentras en: Supabase Dashboard → Settings → Database → Connection string → URI

5 Base de Datos en Supabase

5.1 Ejecutar el DDL en Supabase

- 1 Ir a supabase.com e iniciar sesión en tu proyecto.
- 2 En el panel lateral, ir a SQL Editor.
- 3 Hacer clic en 'New query'.
- 4 Copiar y pegar el contenido de sql/ddl/01_create_tables.sql.
- 5 Hacer clic en Run (o Ctrl+Enter) para ejecutar.
- 6 Repetir con sql/dml/02_seed_data.sql para insertar datos de prueba.
- 7 Verificar en Table Editor que las 13 tablas aparecen correctamente.

5.2 Tablas del sistema

Dominio	Tablas
Usuarios y Acceso	Roles, Users, User_Phones, Document_Types, User_Documents
Personal Clínico	Departments, Specialties, Schedules, Doctors
Infraestructura	Medical_Offices, Speciality_Offices
Pacientes	Guardians, Patients
Atención Médica	Medical_Appointments, Medical_Histories, Medicines, Prescriptions
Auditoría	Audit_Logs

■ **Nota:** La tabla Audit_Logs registra automáticamente cada INSERT, UPDATE y DELETE mediante triggers definidos en el DDL. No requiere configuración adicional.

6 Ejecutar el Backend FastAPI

Con el entorno virtual activo y el archivo `.env` configurado, ejecuta desde la carpeta `backend/`:

```
$ cd backend

$ uvicorn app.main:app --reload
```

Salida esperada en consola:

```
INFO: Will watch for changes in these directories: [...]

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

INFO: Started reloader process with statreload
```

Endpoints disponibles — Módulo Auth

Método	Ruta	Descripción	Auth
GET	/	Mensaje de bienvenida	No
POST	/auth/login	Login → retorna JWT	No
POST	/auth/register	Registrar usuario	No
GET	/auth/users	Listar todos los usuarios	Admin
GET	/auth/users/{id}	Ver usuario por ID	JWT
PATCH	/auth/users/{id}/status	Activar / desactivar usuario	Admin
PATCH	/auth/users/{id}/password	Cambiar contraseña	JWT
PATCH	/auth/users/{id}/username	Cambiar nombre de usuario	JWT

■ **Tip:** La documentación interactiva Swagger está disponible en <http://127.0.0.1:8000/docs>

7 Ejecutar el CLI de Gestión de Usuarios

El archivo `scripts/crud.py` es la interfaz de línea de comandos del sistema. Se conecta directamente a Supabase y no requiere que el backend esté corriendo.

7.1 Instalar dependencias del CLI

```
$ pip install psycpg2-binary bcrypt
```

7.2 Ejecutar el CLI

```
# Desde la raíz del proyecto
```

```
$ python scripts/crud.py
```

7.3 Menú principal

```
=====

HealthFlow - Gestion de Usuarios

=====

1. Listar todos los usuarios

2. Ver usuario por ID

3. Crear nuevo usuario

4. Actualizar usuario

5. Desactivar usuario

6. Ver telefonos y documentos

7. Ver registros de auditoria

8. Salir

=====

Selecciona una opcion: _
```

7.4 Ejemplo — Crear un médico (role_id = 2)

```
Selecciona una opcion: 3
```

```
--- Nuevo Usuario ---
```



```
Username : drlopez

Email : lopez@healthflow.com

Password : .....

Roles: 1=Admin | 2=Doctor | 3=Receptionist | 4=Patient

Role ID : 2

--- Telefono (opcional, Enter para omitir) ---

Telefono : 3001234567

Tipo : Mobile

--- Documento (opcional, Enter para omitir) ---

N Documento : 1098765432

Tipos: 1=CC | 2=CE | 3=Passport | 4=TI

Tipo Doc ID : 1

Usuario creado - ID: 25 | Username: drlopez

Telefono registrado: 3001234567

Documento registrado: 1098765432
```

■ **Nota:** La auditoría se registra automáticamente en Audit_Logs cada vez que se crea, actualiza o desactiva un usuario — tanto desde el CLI como desde el backend FastAPI.

8 Verificación del Sistema

Usa esta lista para confirmar que todo funciona correctamente:

■ `python scripts/crud.py` → opción 1

Muestra la lista de usuarios de Supabase

■ <code>http://127.0.0.1:8000/docs</code>	Swagger UI con todos los endpoints
■ <code>POST /auth/login</code> con credenciales válidas	Retorna un <code>access_token</code> JWT
■ <code>GET /auth/users</code> con el token JWT	Lista de usuarios en JSON
■ CLI opción 7 — Ver auditoría	Registros INSERT/UPDATE/DELETE visibles
■ Crear usuario en CLI → verificar en Swagger	El mismo usuario aparece en ambos lados

9 Solución de Problemas Comunes

■ `ModuleNotFoundError: No module named 'database'`

■ Todos los imports dentro de `app/` deben usar rutas absolutas: `from app.database import ...`

■ `AttributeError: 'Settings' object has no attribute 'DATABASE_URL'`

■ Los atributos en `config.py` están en minúscula. Usar `settings.database_url` (sin mayúsculas).

■ `KeyboardInterrupt` al conectar el CLI a Supabase

■ Agregar `?sslmode=require` al final de la `DATABASE_URL`. Supabase requiere SSL obligatoriamente.

■ `ImportError: email-validator is not installed`

■ Ejecutar en terminal: `pip install 'pydantic[email]'`

■ `500 Internal Server Error` al eliminar un registro

■ El registro tiene dependencias FK. Usar eliminación lógica (`active=False`) en lugar de DELETE físico.